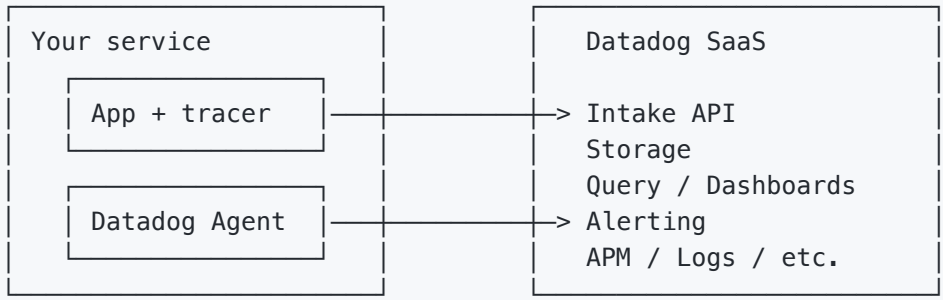
Datadog - Cheat Sheet

Datadog is a SaaS observability platform. Metrics, logs, traces, real-user monitoring, synthetics, security, all in one product. You pay for the convenience of one vendor handling ingestion, storage, and correlation, plus a sharp UI that ties signals together.

The trade-off vs an open-source stack (Prometheus + Grafana + Loki + Tempo) is convenience for cost. Datadog gets expensive fast at scale, especially with high-cardinality custom metrics.

Architecture



- **Datadog Agent**: a process you run on each host or as a DaemonSet in Kubernetes. Scrapes system metrics, collects logs, receives traces, forwards everything to the Datadog SaaS.
- **Library tracers**: per-language libraries (Java agent, Node tracer, etc.) that auto-instrument code and send traces directly or via the Agent.
- **Datadog backend**: SaaS, multi-region (US, EU, AP1, etc.). All your data lives there.

For Kubernetes, the standard setup is the Datadog Operator (or the Helm chart) which runs the Agent as a DaemonSet plus a Cluster Agent for cluster-wide metrics.

The Datadog Agent

The Agent is the universal collector. It does:

- **System metrics** (CPU, memory, disk, network) via integrations.
- **Log collection** from files, journald, Docker, Kubernetes.
- **APM trace forwarding** (your app sends traces to `localhost:8126`, Agent forwards to Datadog).
- **DogStatsD** (UDP statsd server on `localhost:8125` for custom metrics).
- **Live process monitoring** and live container inspection.
- **Network performance monitoring** if enabled.

Config lives at `/etc/datadog-agent/datadog.yaml`. Integrations live in `/etc/datadog-agent/conf.d/`. The Agent ships with hundreds of integrations (Postgres, Kafka, Redis, Nginx, etc.)

that scrape product-specific metrics.

```
# /etc/datadog-agent/datadog.yaml
api_key: <your-key>
site: datadoghq.com           # or datadoghq.eu, us3.datadoghq.com, etc.

logs_enabled: true
apm_config:
  enabled: true

tags:
  - env:prod
  - team:platform
```

What Datadog ingests

A non-exhaustive list of product surfaces.

Product	What it collects
Infrastructure	Host, container, cloud resource metrics
APM	Distributed traces with spans
Logs	Application and system logs
Metrics	Custom metrics from your code
RUM	Browser and mobile user telemetry
Synthetics	Periodic checks (API uptime, browser flows)
Database Monitoring	Per-query DB performance
Network Performance Monitoring	Connection-level network metrics
Cloud Security	Misconfigurations, threats
CI Visibility	Build and test telemetry
Profiling	Continuous CPU/memory profiles

The four you'll touch most as a backend dev: infrastructure (free with Agent), APM, logs, custom metrics.

Metrics

Datadog's metric model is similar to Prometheus, with some differences.

Metric types

Type	Notes
`COUNT`	Monotonic counter. Submit with `dd.count(name, n)`. Aggregated by sum.
`RATE`	Pre-divided rate (count per unit time). Less common.
`GAUGE`	Point-in-time value. Submit with `dd.gauge(name, v)`.
`HISTOGRAM`	Aggregated to count/avg/median/p95/max on the Agent before send.
`DISTRIBUTION`	Global distribution, aggregated server-side. Supports cross-host percentiles.

Histograms and distributions both produce percentile metrics. Distributions cost more but give correct global percentiles. Histograms are cheaper but per-host (you can't compute global p99 by averaging per-host p99s).

Tagging

Every metric carries tags. Tags slice metrics in dashboards and alerts.

```
http.requests.count {env:prod, service:orders, status:200, method:GET}
```

Datadog's tags look like Prometheus labels with a different syntax (`key:value` with colons).

Three unified tags are special: `env`, `service`, `version`. They power the service catalog, correlation between APM and logs, and most pre-built dashboards. Always set them.

Custom metrics from Java

Three paths.

1. DogStatsD client. UDP to the Agent.

```
StatsDClient statsd = new NonBlockingStatsDClientBuilder()
    .prefix("orders")
    .hostname("localhost")
    .port(8125)
    .build();

statsd.incrementCounter("placed", "status:success", "region:us");
statsd.recordExecutionTime("place.latency", durationMs, "status:success");
statsd.gauge("queue.depth", queueSize);
statsd.distribution("payment.amount", amount.doubleValue());
```

2. Micrometer Datadog registry. If you already use Micrometer, swap the registry.

```
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-datadog</artifactId>
</dependency>
```

```
management:
  metrics:
    export:
      datadog:
        api-key: ${DD_API_KEY}
        application-key: ${DD_APP_KEY}
        step: PT30S
```

Micrometer counters become Datadog counts. Micrometer timers become histograms. No code changes.

3. Datadog Java tracer's `Metrics` API. Bundled with `dd-java-agent`.

For most apps, Micrometer + Datadog registry is the cleanest path. Switching observability backends later only takes a registry swap.

Logs

Shipping logs

Three options:

- **Agent tail.** Configure the Agent to read log files. Standard for VM and Kubernetes deploys.
- **Direct API.** Use `dd-java-agent` log injection or a Logback appender to ship straight to Datadog.
- **Sidecar.** A sidecar container reads stdout and forwards. Common in orchestrators that don't run the Agent on every host.

```yaml
# Agent config to tail an app log
logs:
  - type: file
    path: /var/log/myapp/*.log
    service: orders
    source: java
```

Pipelines and parsing

Datadog processes logs through pipelines: parse, enrich, mask, route. A Java log line:

```
2025-05-24 12:00:00.123 INFO  [trace_id=abc def=789] OrderService - placed order id=42
```

A Grok parser in the pipeline extracts `trace_id`, `span_id`, log level, message. After parsing, the structured fields become searchable.

JSON logs skip the parsing step. Define the format up front and Datadog indexes every field automatically.

```json
{
  "timestamp": "2025-05-24T12:00:00Z",
  "level": "INFO",
  "service": "orders",
  "trace_id": "abc123",
  "message": "placed order",
  "order_id": 42
}
```

Indexes and exclusions

Logs cost money to index. By default, Datadog indexes everything. Use exclusion filters to drop noisy DEBUG logs at ingest. Use rehydration if you need to query them later (they live in S3).

Logs to metrics

Generate a metric from a log query without writing code:

```
service:orders status:error  -> orders.errors.count
```

Useful when you don’t control the app but need a metric for alerting.

APM and distributed tracing

Setup with `dd-java-agent`

The Datadog Java tracer attaches at JVM start.

```
java -javaagent:/path/to/dd-java-agent.jar \
     -Ddd.service=orders \
     -Ddd.env=prod \
     -Ddd.version=1.4.2 \
     -jar myapp.jar
```

It auto-instruments common libraries: Spring, JDBC, HTTP clients (Apache, OkHttp), Kafka, gRPC, AWS SDK, Redis, MongoDB, and more. No code changes for most stacks.

Manual instrumentation

When you need custom spans:

```
import datadog.trace.api.Trace;

public class OrderService {

    @Trace(operationName = "order.place")
    public void place(Order order) {
        // span auto-starts and ends around this method
    }
}
```

Or with the OpenTelemetry API (Datadog supports OTel):

```
Tracer tracer = GlobalOpenTelemetry.getTracer("orders");
Span span = tracer.spanBuilder("place-order")
    .setAttribute("customer.id", order.customerId())
    .startSpan();
try (Scope scope = span.makeCurrent()) {
    // work
} finally {
    span.end();
}
```

Trace propagation

Headers ride with HTTP and gRPC calls. Datadog's native format uses `x-datadog-trace-id` and `x-datadog-parent-id`. The tracer also supports W3C `traceparent` and B3 headers. Configure with `dd.trace.propagation.style`.

Service map and trace explorer

Once traces flow, Datadog builds a service map automatically. The trace explorer lets you query traces by service, operation, latency, tag, error status.

Linking: each trace span shows correlated logs (same `trace_id`), profiler samples (if continuous profiling is on), and infrastructure metrics for the host.

Dashboards

Drag-and-drop widgets backed by Datadog query expressions.

```
sum:http.requests.count{env:prod,service:orders}.as_rate()
avg:system.cpu.user{host:web-*} by {host}
sum:trace.servlet.request.errors{service:orders}.as_count()
  / sum:trace.servlet.request.hits{service:orders}.as_count()
```

Query structure: `aggregator:metric.name{tag filter} by {grouping}.modifier()`.

Common modifiers:

- `.as_rate()` - per-second rate
- `.as_count()` - sum over the interval
- `.rollup(avg, 60)` - bucket the data
- `.top(5, 'last', 'mean')` - top 5 by mean

Template variables work like Grafana: `$service`, `$env`. Reuse a dashboard across services with one click.

Widget types: time series, query value, top list, table, heatmap, distribution, log stream, trace search, service map, hostmap (geographic). Notebooks are like dashboards but linear for writeups.

Monitors (alerting)

Monitor type	Triggers on
Metric	A metric query crossing a threshold
Anomaly	Metric deviates from its baseline
Forecast	A metric will hit a threshold soon
Outlier	One host or tag value differs from peers
Composite	Combination of other monitors
Log	Log query count
APM	Trace error rate, latency, throughput
Process	Process count on a host
Service Check	Custom check from the Agent
Network	Connection-level conditions
Synthetic	Synthetic test failures
Watchdog	ML-detected anomalies, no config needed

Metric monitor example

```
avg(last_5m):
  sum:trace.servlet.request.errors{service:orders}.as_count()
  / sum:trace.servlet.request.hits{service:orders}.as_count()
  > 0.01
```

Configure:

- **Alerting condition**: above (over a threshold), below, anomaly, etc.
- `for` duration: how long the condition must hold.
- **Notification channels**: Slack, PagerDuty, OpsGenie, email, webhook.
- **Renotification**: how often to remind if still firing.
- **Recovery**: when to close the alert.

The notification message uses Datadog's templating:

```
{{#is_alert}}
Error rate on {{service.name}} hit {{value}}%.
Runbook: https://wiki/runbooks/orders-errors
{{/is_alert}}
```

Composite monitors

Combine signals so a single page only fires when multiple conditions hold:

```
(monitor_id_1 && monitor_id_2) || monitor_id_3
```

Useful for “alert only when error rate is high AND traffic is non-trivial”.

SLOs

SLOs in Datadog combine an SLI query and an objective.

- **Metric-based SLO**: count “good” events vs total events.
- **Monitor-based SLO**: percentage of time the underlying monitor was healthy.

```
SLO: orders-availability
  Type: Metric
  Numerator:   sum:trace.servlet.request.hits{service:orders,!status:5xx}.as_count()
  Denominator: sum:trace.servlet.request.hits{service:orders}.as_count()
  Target:      99.9% over 30d
```

Datadog shows error budget burn-down. Fast-burn alerts fire when you’re consuming budget too quickly.

Tagging strategy

Tags are how Datadog correlates everything. Pick them carefully.

The three unified tags Datadog expects on every signal:

- `env` : dev, staging, prod
- `service` : the service name (orders, users, etc.)
- `version` : the deployed release version

Beyond unified tags, useful tags:

- `team` : who owns it. Useful for routing alerts.
- `region` : us-east-1, eu-west-1.
- `cluster` : k8s cluster or AZ.

- `tenant`: per-customer slicing (mind cardinality).

Bad tags to avoid: per-request IDs (`user_id`, `request_id`, `trace_id`), unbounded URL paths, anything with high cardinality. Same lesson as Prometheus.

Cost management

Datadog's pricing has many dimensions. The big ones for backend teams:

Surface	What you pay for
Hosts	Per host per month
Containers	Per container per month (beyond a free quota per host)
Custom metrics	Per unique tag combination ("custom metric"), 100 included per host
APM	Per host with APM enabled
Logs	Per GB ingested + per million indexed
Profiling	Per host with continuous profiling enabled
RUM, Synthetics, etc.	Per session, per check

The metric one is the silent killer. A custom metric `orders.placed{env, service, status, method, endpoint, region}` with 2 envs * 5 services * 10 statuses * 5 methods * 50 endpoints * 4 regions = 100,000 custom metrics. That alone is a five-figure monthly bill.

Cardinality control:

- Drop labels you don't query on.
- Use `--tag-filter` on the Agent to strip noisy tags.
- Set custom metric quotas per team.
- Audit `metric_count` per metric name in the Metrics Summary.

Datadog vs OSS observability

Aspect	Datadog	Prometheus + Grafana + Loki + Tempo
Setup	Install Agent, point at SaaS	Self-host each component
Operations	Vendor handles	Your team handles
Cost model	Per-host, per-metric, per-GB	Infrastructure cost only
Correlation across signals	First-class, automatic	Manual (matching labels and trace IDs)
Vendor lock-in	High (proprietary query language, dashboards)	Low (mostly open standards)
Cost at large volumes	Often higher	Often lower

Many teams start on Datadog for speed, then migrate to the open-source stack when bills get big. Some run both: Datadog for APM and incidents, Prometheus for high-cardinality custom metrics where Datadog cost would be unbearable.

Best practices

- **Set `env`, `service`, `version` on everything.** Datadog's correlation depends on it.
- **Use Micrometer + the Datadog registry** for custom metrics. Decouples your app from Datadog-specific APIs.
- **Use distribution metrics for percentiles** you need across hosts.
- **Ship structured JSON logs** so Datadog parses fields automatically.
- **Drop noisy DEBUG logs at the Agent** with log exclusion filters. Indexing them later costs real money.
- **Use the unified service tagging** to tie APM, logs, and metrics together by clicking once.
- **Tag every dashboard and monitor with team and service.** Helps when someone asks "who owns this?".
- **Set monitor escalation paths and runbook links** in the alert message. Pages without context are pages people learn to ignore.
- **Use composite monitors** to avoid alerting when traffic is too low for the rate to be meaningful.
- **Set custom-metric budgets per team.** Otherwise one well-meaning developer adds a tag to a metric and blows the monthly bill.

Common gotchas

- **Custom metric explosion.** A tag with thousands of values multiplies your custom metric count. Each unique combination is its own billable metric.
- **Histograms can't aggregate across hosts.** A global p99 from per-host histograms is wrong. Use distribution metrics.
- **Missing unified tags.** Without `env`, `service`, `version`, APM and logs won't correlate cleanly.
- **Trace sampling losing the trace you wanted.** Default is head-based sampling. Sample rate decisions happen at the trace start. Configure rules for critical paths.
- **Agent on the wrong host.** Container hostnames vs node hostnames can confuse the host count. Verify with the Infrastructure list.
- **`dd-java-agent` version drift.** Old tracer versions miss new auto-instrumentations. Update on a schedule.
- **`@Trace` on too many methods.** Tracing every getter makes the trace explorer noisy and adds overhead. Trace at meaningful boundaries.
- **Indexes that cover everything.** The default log index keeps everything for 15 days. For high-volume services, write exclusion filters that drop the chatty 90%.
- **Forgetting to set log retention.** Indexed logs default to 15-day retention. Some compliance scenarios need longer. Some workloads need shorter.
- **Monitor evaluation window too short.** A 1-minute window catches every blip. 5-15m windows for SLO-level monitors.
- **Mixed metric types.** Submitting the same metric as count from one host and gauge from another corrupts the data. Standardize.
- **DogStatsD UDP packet drops.** Under load, UDP packets to the Agent can drop. Use the buffered client and monitor the dropped count.

Quick reference

Concept	Key fact
Agent	Per-host or per-cluster collector. Localhost endpoints for traces (8126) and StatsD (8125).
Unified tags	`env`, `service`, `version`. Always set them.
Custom metric	A unique combination of name + tags. Each is billable.
Distribution metric	Pre-aggregated histogram with correct global percentiles.
`dd-java-agent`	`-javaagent` jar that auto-instruments Spring, JDBC, Kafka, etc.
`@Trace`	Annotation for custom spans
Logs to metrics	Generate a metric from a log query without code
Composite monitor	Combine multiple monitors with logic
SLO	Numerator-over-denominator query plus a target
Anomaly monitor	Trigger when metric deviates from baseline
Watchdog	ML-detected anomalies with no config
`.as_rate()`	Convert a count metric to per-second
`.as_count()`	Sum a metric over the interval
Service map	Auto-generated from traces
Cost levers	Hosts, custom metrics, log GB and indexed events